

Implementation of Google DeepMind Learning Mesh-Based Simulation with Graph Neural Networks (ICLR 2021)

1. Introduction

Mesh-based simulations are the heart of modeling complex physical systems in modern science and engineering problems. They encompass powerful numerical integration methods, which put most developers on a resolution tradeoff between accuracy and efficiency. However, high-dimensional problems are usually computationally intensive and time-consuming to run, and they are often tuned for a specific case study. This means that each new run typically requires the same substantial amount of computational resources and patience.

The mesh-based graph neural network is a framework for learning mesh-based simulations using graph neural networks (GNNs), published by Google DeepMind in 2021. GNNs are architectures that extend deep learning to data represented as graphs. Unlike grid data, mesh-based simulations naturally form graph structures due to their connection. The result of implementing the workflow shows that it can accurately predict the flow around a sphere in a cylinder. In addition, the model is very efficient for prediction but has the downside of a long training time.

2. Model Architecture: Encode-Process-Decode

The model follows the **Encode–Process–Decode** paradigm, a widely used design in physics-based GNNs.

2.1 Encoder

The encoder transforms raw node, edge, and global features into latent representations. In this model:

- Each feature is passed through an MLP.
- The output latent dimension is 128.
- Encoded features form the initial graph state for message passing.

2.2 Processor (Message Passing)

The processor is the core of the network.

- It performs **15 message-passing steps**.
- Each step updates nodes and edges by aggregating information from neighbors.
- The processor includes **2 MLP layers** per update block.
- This enables learning local interactions and long-range dependencies in the mesh.

2.2 Decoder

After message passing, the decoder maps latent node states to the target output.

- Output size is defined by `params['size']`.
- Produces predicted quantities such as **velocity**, **pressure**, or other simulation fields.

3. Training Pipeline

3.1 Dataset Preparation

1. The raw mesh-based simulation data is loaded.
2. Target fields (e.g., velocity) are appended.
3. Historical features are added depending on params['history'].
4. Noise augmentation is applied with parameters:
 - noise_scale
 - noise_gamma
5. Non-floating tensors (like node indices) are cast to tf.int32.

3.2 Optimization

Employed **Adam** optimizer with a learning rate of **Exponential Decay** starting at $1e-4$. NOTE: The initial 1000 steps act as a **normalization warm-up**

3.3 Training Loop

Implemented with “@tf.function” for GPU acceleration

4. Evaluation and Rollout Generation

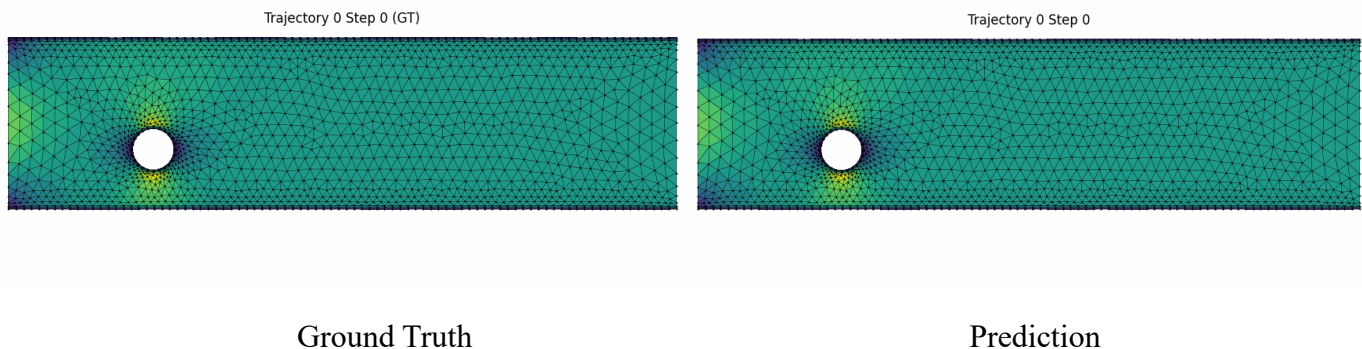
GNNs for physical simulation are evaluated by "rollout" performance.

4.1 Rollout Structure

A rollout consists of:

- A sequence of meshes over time (positions, connectivity)
- Ground truth velocity fields
- Predicted velocity fields from the model

4.2 Visualization



5. Key Contributions

- Implemented workflow in TensorFlow v2 and Sonnet v2 (Original: TensorFlow v1 and Sonnet v1)
- Employed `@tf.function` to improve GPU utilization
- Wrote beginner-friendly comments to help explain the workflow. Comments are intended to be updated on the original code.
- Employed 200,000 as opposed to 10M in the original workflow and got a decent prediction. This reduction was due to the estimated training time.

6. Conclusion

This work successfully implemented the Google DeepMind *Learning Mesh-Based Simulation with Graph Neural Networks* framework using TensorFlow v2 and Sonnet v2, demonstrating the effectiveness of graph neural networks for modeling complex physical systems. By leveraging the Encode–Process–Decode architecture, the model was able to learn local and long-range interactions on unstructured meshes and generate accurate rollout predictions for fluid flow around a sphere in a cylinder.

The results highlight the key strength of mesh-based GNNs: once trained, they provide fast and efficient inference compared to traditional numerical solvers, making them well-suited for repeated simulation tasks. Despite using significantly fewer training steps (200,000 compared to the original 10 million), the model achieved reasonable predictive performance, confirming the robustness of the architecture even under constrained computational budgets.

However, the implementation also reinforces an important limitation of learning-based simulation models: training remains computationally expensive and time-consuming, especially for high-dimensional problems. While GPU acceleration and TensorFlow's `@tf.function` improved performance, the upfront training cost is still substantial. This highlights a trade-off between training expenses and inference efficiency.

Overall, this project demonstrates that mesh-based GNNs are a powerful alternative to classical solvers for physical simulation, particularly in scenarios requiring rapid prediction after training. Future work may explore improved normalization strategies, reduced training time through curriculum learning, or hybrid approaches that combine learned models with traditional physics-based solvers to further enhance efficiency and generalization.